

Aurora — Proteção da branch `main` com PR + CI verde (Guia Completo)

Guia didático (passo a passo + teoria + troubleshooting) do que fizemos no repositório **aurora-platform** para ativar **branch protection** exigindo **pull request** e **status checks** (CI) verdes.

Use este material como **apostila** para a turma e como **runbook** do time.

1) Contexto e fundamentos

1.1 O que é “branch protection”

Regras que impedem commits diretos numa branch (ex.: `main`) e exigem critérios para merge (review, CI verde, histórico linear etc.).

1.2 Por que exigir PR + checks verdes?

- **Qualidade:** cada mudança passa por revisão e por uma suíte automática mínima (lint, build, test).
- **Estabilidade:** “Require branches to be up to date” garante que o PR foi testado contra o último commit da `main`.
- **Auditoria:** histórico de quem aprovou, que checks passaram, qual PR introduziu a mudança.

1.3 Conceitos que usamos

- **Pull Request (PR):** fluxo de revisão e aprovação antes do merge em `main`.
 - **Status checks:** resultados de jobs do GitHub Actions (ou outros provedores) que o GitHub conhece e pode “exigir”.
 - **Contexts:** o “nome” de cada check. Em Actions geralmente é o **nome do job** (`lint`, `build`, `test`).
 - **Rules vs Branch protection:** usamos a **Branch protection rule** clássica na própria `main` (não o recurso novo de *Repository rules*).
-

2) Pré-requisitos e decisões do projeto

- Repositório: **aurora-platform** (monorepo futuro; por enquanto repositório simples).
- Porta padrão documentada: **3100** (README atualizado via PR).
- CI no GitHub Actions com **3 jobs:** `lint`, `build`, `test`.
- A proteção exigirá **PR** e **os 3 checks verdes**.

⚠ O GitHub só lista checks que **rodaram pelo menos uma vez na última semana**. Por isso primeiro criamos o workflow, abrimos um PR e **deixamos a pipeline rodar**, para depois selecionar os checks na regra.

3) Criando o CI (GitHub Actions)

3.1 Arquivos

Crie o workflow em `/.github/workflows/ci.yml`.

3.1.1 Versão “segura” (funciona mesmo sem `package.json`)

Essa versão **detecta** se existe `package.json`. Se não existir, registra “skipped” e **passa** (útil enquanto o repo ainda não tem app Node). Quando o projeto existir, os jobs passam a rodar de verdade.

```
name: CI

on:
  pull_request:
    branches: [ main ]
  push:
    branches: [ main ]

permissions:
  contents: read

concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Detect Node project
        id: detect
        run: |
          if [ -f package.json ]; then echo "has_node=true" >>
$GITHUB_OUTPUT; else echo "has_node=false" >> $GITHUB_OUTPUT; fi
      - uses: actions/setup-node@v4
        if: steps.detect.outputs.has_node == 'true'
        with: { node-version: 20, cache: npm }
      - run: npm ci
        if: steps.detect.outputs.has_node == 'true'
      - run: npm -ws run lint --if-present
        if: steps.detect.outputs.has_node == 'true'
      - name: No Node project – lint skipped (pass)
        if: steps.detect.outputs.has_node != 'true'
        run: echo "No package.json at repo root. Lint skipped."

  build:
    runs-on: ubuntu-latest
    needs: lint
```

```

steps:
  - uses: actions/checkout@v4
  - name: Detect Node project
    id: detect
    run: |
      if [ -f package.json ]; then echo "has_node=true" >>
$GITHUB_OUTPUT; else echo "has_node=false" >> $GITHUB_OUTPUT; fi
  - uses: actions/setup-node@v4
    if: steps.detect.outputs.has_node == 'true'
    with: { node-version: 20, cache: npm }
  - run: npm ci
    if: steps.detect.outputs.has_node == 'true'
  - run: npm -ws run build --if-present
    if: steps.detect.outputs.has_node == 'true'
  - name: No Node project – build skipped (pass)
    if: steps.detect.outputs.has_node != 'true'
    run: echo "No package.json at repo root. Build skipped."

test:
  runs-on: ubuntu-latest
  needs: build
  steps:
    - uses: actions/checkout@v4
    - name: Detect Node project
      id: detect
      run: |
        if [ -f package.json ]; then echo "has_node=true" >>
$GITHUB_OUTPUT; else echo "has_node=false" >> $GITHUB_OUTPUT; fi
    - uses: actions/setup-node@v4
      if: steps.detect.outputs.has_node == 'true'
      with: { node-version: 20, cache: npm }
    - run: npm ci
      if: steps.detect.outputs.has_node == 'true'
    - run: npm -ws run test --if-present -- --ci
      if: steps.detect.outputs.has_node == 'true'
    - name: No Node project – test skipped (pass)
      if: steps.detect.outputs.has_node != 'true'
      run: echo "No package.json at repo root. Test skipped."

```

3.1.2 Versão “real” (quando o projeto Node existir)

```

name: CI
on: [push, pull_request]
jobs:
  lint:
    runs-on: ubuntu-latest
    steps: [ { uses: actions/checkout@v4 }, { uses: actions/setup-node@v4,
with: { node-version: 20, cache: npm } }, { run: npm ci }, { run:
npm -ws run lint --if-present } ]
    build:

```

```

runs-on: ubuntu-latest
needs: lint
steps: [ { uses: actions/checkout@v4 }, { uses: actions/setup-node@v4,
with: { node-version: 20, cache: npm } }, { run: npm ci }, { run:
npm -ws run build --if-present } ]
test:
runs-on: ubuntu-latest
needs: build
steps: [ { uses: actions/checkout@v4 }, { uses: actions/setup-node@v4,
with: { node-version: 20, cache: npm } }, { run: npm ci }, { run:
npm -ws run test --if-present -- --ci } ]

```

Padrão de nomes: mantivemos os jobs como `lint`, `build`, `test` — esses nomes viram os **contexts** dos checks que serão marcados como obrigatórios.

3.2 Disparando os checks pela primeira vez

1. Abra um **PR** (edite o `README.md`, por exemplo, e crie uma branch nova).
2. Acompanhe a aba **Checks** do PR; os jobs devem rodar e ficar **verdes**.

4) Configurando a proteção da branch `main`

1. Acesse **Settings** → **Branches** → **Add rule** (ou **Edit** se já existir) e informe o padrão `main`.
2. Marque **Require a pull request before merging**.
3. (Opcional) **Require approvals** = 1.
4. Marque **Require status checks to pass before merging**.
5. Em **Search for status checks...**, adicione os chips: `lint`, `build`, `test`.
6. Marque **Require branches to be up to date before merging**.
7. (Recomendado) **Do not allow bypassing the above settings**.
8. Mantenha **Allow force pushes** e **Allow deletions** desmarcados.
9. (Opcionais) **Require conversation resolution**, **Require linear history**, **Require signed commits** (se seu time assina commits), **Require deployments to succeed** (se usar Environments).
10. **Save changes**.

■ Em alguns repositórios o campo de busca mostra apenas `lint`, `build`, `test` (sem prefixo `CI /`). Se não aparecer sugestão, **digite o nome e clique na opção "Add ..."** para transformar em um chip.

5) Testando a proteção

1. Abra um PR simples (ex.: editar o README).
2. Verifique que aparece a faixa **"Required statuses must pass: lint, build, test"**.
3. O botão **Merge** só libera quando os checks estiverem **verdes** (e com **review**, se for exigido).

6) Problemas comuns e como resolver (Troubleshooting)

6.1 “No required checks / No checks have been added”

Causa: os jobs **nunca rodaram recentemente**.

Solução: abra um PR e deixe a pipeline rodar **ao menos uma vez**; então volte à regra e selecione os checks.

6.2 “CI / lint failing” em repositório vazio

Causa: workflow tentou `npm ci` sem `package.json`.

Solução: use a versão “segura” do `ci.yml` (detecção de Node) até o projeto existir.

6.3 Lista de checks não mostra sugestões

Solução A: digite `lint`, `build`, `test` e clique na opção **Add ‘...’**.

Solução B: faça merge do PR do `ci.yml` para `main`, rode o Actions em `push` e retorne — as sugestões costumam aparecer.

6.4 PR autor não consegue aprovar

Causa: com “Require approvals” ligado, o autor do PR **não pode** aprovar o próprio PR.

Solução: peça aprovação a outro membro ou **desmarque temporariamente** “Require approvals” para configurar o projeto inicial.

6.5 “Checking for the ability to merge automatically...” infinito

Solução: recarregue a página; clique em **Update branch** (se aparecer) ou atualize a branch localmente (`git merge origin/main` ou `git rebase`) e *push* novamente. Re-rode os checks se necessário.

6.6 Actions desabilitado

Solução: Settings → Actions → General → **Allow all actions and reusable workflows**.

6.7 Nomes dos jobs mudaram depois

Solução: atualize os **Required status checks** na regra para refletir os novos contexts.

6.8 Forçar seleção via CLI

Se a UI estiver *bugada*, é possível gravar os checks exigidos com a **GitHub CLI**:

```
# troque <owner>/<repo> pelo seu
gh api -X PUT -H "Accept: application/vnd.github+json" \
  repos/<owner>/<repo>/branches/main/protection \
  -f enforce_admins=true \
  -f required_pull_request_reviews='{"required_approving_review_count":0}' \
  -f required_status_checks='{"strict":true,"contexts":
```

```
["lint","build","test"]}' \
-f restrictions='null'
```

7) Boas práticas e justificativas

- **Jobs pequenos e nomeados:** `lint`, `build`, `test` como unidades claras; ajudam na leitura e seleção.
- **Up-to-date antes do merge:** reduz “flakiness” — você testa exatamente o que será mergeado.
- **Evitar bypass:** marque “Do not allow bypassing...” para aplicar regras também a administradores.
- **Histórico limpo:** use **Squash and merge** para PRs pequenos (docs, fixes) e **Merge commit** apenas quando fizer sentido manter múltiplos commits.
- **PR template:** padronize descrição (contexto, o que muda, impacto/risco, testes, follow-ups).

Exemplo de PR (usado por nós):

Contexto: porta padrão passou a **3100**; README mencionava **3000**.

O que muda: atualiza README `3000 → 3100`; sem mudança de código.

Impacto: baixo. **Teste:** visualizar README; **Follow-ups:** alinhar `.env/compose/main.ts` quando existirem.

8) Extensões recomendadas

- **CODEOWNERS** para exigir review de áreas específicas.
- **Required conversation resolution** para evitar comentários pendentes.
- **Signed commits** se a organização adota GPG/SSH signing.
- **Required deployments** com *Environments* (ex.: garantir deploy de staging antes do merge).
- **Outros checks** (SAST/secret scanning) como **GitGuardian Security Checks**.

9) Checklist final (para auditar o repositório)

- ☐ Existe `/.github/workflows/ci.yml` com jobs `lint`, `build`, `test`.
- ☐ Um PR rodou os três checks nos últimos 7 dias.
- ☐ **Settings → Branches → main:**
- ☐ **Require a pull request before merging**
- ☐ (Opcional) **Require approvals (≥1)**
- ☐ **Require status checks to pass** com `lint`, `build`, `test`
- ☐ **Require branches to be up to date**
- ☐ **Do not allow bypassing the above settings**
- ☐ **Allow force pushes:** desmarcado
- ☐ **Allow deletions:** desmarcado
- ☐ Fluxo de **Squash and merge** definido para PRs simples.
- ☐ PR template adotado.

10) Anotações de aula com referência visual

Não é necessário anexar imagens para seguir o guia, mas durante a aula mostramos as seguintes telas: - **Tela 1 - Branch protection: lista de regras** (botão **Edit** ao lado da `main`). - **Tela 2 - Formulário de edição** (seções *Require a pull request...*, *Require status checks...*, *Require branches to be up to date*). - **Tela 3 - Campo "Search for status checks..."** (onde adicionamos `lint`, `build`, `test`). - **Tela 4 - PR com checks rodando** (aba **Checks** com `lint`, `build`, `test`). - **Tela 5 - PR liberado para merge** (todos os checks verdes; usamos **Squash and merge**).

11) Próximos passos

- Iniciar o monorepo **aurora-platform** (workspaces) e criar o microserviço **users** com **NestJS + TypeORM + Postgres**.
 - Expandir o CI para rodar **lint/build/test** em cada workspace.
 - Adicionar *lint rules* (ESLint/Prettier) e cobertura de testes.
-

Fim do guia. Se algo quebrar, volte à seção **Troubleshooting** ou abra um PR de teste e verifique a aba **Checks** — ela sempre conta a história do que o GitHub está exigindo para liberar o merge.